



LOVELY
PROFESSIONAL
UNIVERSITY

School of Computer Science and Engineering

Lovely Professional University

Phagwara, Punjab (India)

Topic: - Cloud-native security monitoring and threat intelligence platform

Subject: - INT377 (CLOUD COMPUTING AND DEVOPS ESSENTIALS)

Student Name	Cahal Agarwalla
Registration No.	12309960
Section	3OM57
Roll no.	17
Faculty Guide	Dr. Chavi Ralhan

Declaration

I, Cahal Agarwalla (Registration No. 12309960), a student of B.Tech Computer Science & Engineering, 6th Semester at Lovely Professional University, Phagwara, Punjab, hereby declare that the project report entitled "ThreatShield — Cloud-Native Cybersecurity Monitoring & Threat Intelligence Platform" is a record of my own work carried out under the guidance of Ma'am Chavi Ralhan.

I further declare that this project has not been submitted elsewhere for any other degree or examination, and that all sources of information used have been duly acknowledged.

Cahal Agarwalla

Registration No.: 12309960

B.Tech CSE — 6th Semester

Lovely Professional University

Date: 26th May, 2026

Abstract

In today's interconnected world, internet-facing systems face a relentless barrage of automated attacks — SSH brute-force scans, credential-stuffing bots, and opportunistic port scanners operate around the clock across the global internet. Traditional perimeter-based defences attempt to block this traffic at the firewall level, but they provide no visibility into attacker behaviour, the tools they use, or the geographic origins of attacks. Without this intelligence, defenders remain reactive.

ThreatShield is a cloud-native cybersecurity monitoring and threat intelligence platform built to address this gap. Rather than relying solely on prevention, ThreatShield adopts an active defence posture by deploying honeypots — deliberately exposed decoy systems — in front of the internet. Any interaction with these decoys is, by definition, hostile. Every connection attempt, failed credential, and command executed is captured and turned into actionable threat intelligence.

The platform is built on a modern DevOps architecture. A Terraform script provisions an AWS EC2 t2.micro instance running Ubuntu 22.04 in the ap-south-1 (Mumbai) region. On this instance, Docker Compose orchestrates six containers: a Cowrie SSH/Telnet honeypot, a Flask-based fake corporate web-login portal, a FastAPI backend with asynchronous PostgreSQL, a Prometheus metrics collector, and a Grafana observability dashboard. The React-based SOC (Security Operations Centre) frontend is deployed separately on Vercel's global edge network and receives live attack data via WebSockets from the backend.

A GitHub Actions CI/CD pipeline automates code quality checks (Ruff for Python, ESLint for React), backend testing with a live PostgreSQL service container, Aqua Security Trivy vulnerability scanning, and Docker image builds on every push to the main branch. The entire infrastructure is defined as code, making the system fully reproducible.

Within hours of deployment, ThreatShield captured over 65 real-world attack events from 27 unique attackers across 19 countries, demonstrating the platform's effectiveness as a threat intelligence gathering tool.

Table of Contents

1. Introduction	7
2. Profile of the Problem & Rationale	9
3. Existing System	11
4. Problem Analysis	13
5. Software Requirement Analysis	16
6. System Design & Architecture	19
7. Testing	25
8. Implementation	28
9. Project Legacy	33
10. User Manual	35
11. Source Code & System Snapshots	37
12. Bibliography	40

Chapter 1 — Introduction

1.1 Background

The modern internet is a hostile environment. Automated scanners probe every public IP address for open ports within minutes of assignment. Once an open SSH port, a web login form, or a Telnet endpoint is found, credential-stuffing bots immediately begin testing username–password combinations harvested from prior data breaches. These attacks run at machine speed — thousands of attempts per hour — and originate from botnets spread across dozens of countries simultaneously.

For organisations running internet-facing servers, this is not a theoretical risk — it is a constant, measurable reality. The question is not whether attacks will occur, but how quickly defenders can detect, understand, and respond to them. Security Operations Centres (SOCs) are built precisely for this purpose: centralised teams and tools that monitor events, classify threats, and coordinate responses in real time.

However, building a SOC that captures real attacker behaviour — not simulated data — requires a specific type of system: a honeypot. A honeypot is a deliberately vulnerable decoy that mimics a real production service. It has no legitimate users, so every single interaction it receives is an attack. This makes honeypot telemetry extraordinarily high-signal data, free from the false positives that plague traditional intrusion detection systems.

1.2 Project Overview

ThreatShield is a complete, cloud-native platform that deploys honeypots at internet scale and provides a real-time SOC dashboard for monitoring the captured telemetry. The system integrates five major engineering domains:

- Cloud Infrastructure: AWS EC2 provisioned via Terraform (Infrastructure as Code)
- Container Orchestration: Docker Compose managing six microservices on a single instance
- Active Deception: Cowrie SSH/Telnet honeypot and a custom fake corporate web portal
- Real-Time Backend: Asynchronous FastAPI with PostgreSQL and WebSocket streaming
- DevSecOps: GitHub Actions CI/CD with security scanning via Aqua Trivy
- Observability: Prometheus metrics collection and auto-provisioned Grafana dashboards

The frontend SOC dashboard is deployed on Vercel's edge network and connects to the backend via WebSocket for sub-second live attack feeds, interactive world maps showing attacker geolocations, and timeline charts of attack frequency.

1.3 Objectives

1. Deploy a cloud-native honeypot infrastructure on AWS that captures real-world attack telemetry from the open internet.
2. Build a high-performance, asynchronous backend that ingests honeypot logs, classifies threats by severity, and stores structured data in PostgreSQL.
3. Develop a real-time SOC dashboard that streams live attack events to analysts via WebSockets, displays attacker geolocations on an interactive world map, and presents trend analytics via charts.
4. Implement a complete DevSecOps CI/CD pipeline that automates linting, testing, vulnerability scanning, and Docker image builds on every code push.
5. Establish full observability of the backend infrastructure through Prometheus metrics collection and auto-provisioned Grafana dashboards.
6. Ensure the entire system is Infrastructure as Code (IaC) — reproducible, version-controlled, and deployable from a single Terraform command.

1.4 Scope

This project covers the full stack from cloud provisioning to frontend display. It is intended as a real operational threat intelligence platform rather than a simulation. The honeypots deployed are genuine — they received and logged real attacks from internet scanners within hours of deployment. The platform is accessible publicly at: <https://threat-shield-ai-cloud-native-cyber.vercel.app> and the complete source code is available at <https://github.com/CaHHaL/ThreatShield-Cloud-Native-Cyber-Defense-Monitoring-Platform>

Chapter 2 — Profile of the Problem & Rationale

2.1 Problem Statement

Internet-facing systems are attacked continuously. SSH brute-force, web credential stuffing, and port scanning are not rare events — they are background noise on the modern internet. Despite this, most organisations have no systematic way to study this attack traffic in detail. Firewall logs show blocked connections but reveal nothing about attacker intent, the tools they use, or the commands they attempt to execute once inside a compromised system.

The core problem is a lack of actionable threat intelligence at the infrastructure level. Security teams need answers to questions such as: Which countries are the most active sources of attacks? Which credential combinations are most commonly attempted? How quickly do automated scanners find a newly exposed service? What commands does an attacker run immediately after gaining a foothold? Without a honeypot, these questions cannot be answered from production logs alone.

2.2 Why Honeypots?

A honeypot is placed in front of sensitive systems specifically to intercept and study attackers before they reach production infrastructure. Unlike an intrusion detection system (IDS) that analyses traffic passing through a network, a honeypot actively invites connections. Because honeypots have no legitimate users — no developer SSH-es into a honeypot, no employee logs into a fake corporate portal — every single event they record is an attack.

This makes honeypot data uniquely valuable: it has zero false positives by design. Every IP address that connects to ThreatShield's Cowrie honeypot is an attacker. Every username–password pair it tries is a harvested credential from some prior breach. Every command it runs reveals reconnaissance tactics, persistence techniques, or malware deployment patterns. This is intelligence that no amount of firewall log analysis can provide.

Monitoring honeypots is equally important. A honeypot that logs data to a file with no analysis provides raw telemetry, but not intelligence. ThreatShield transforms raw honeypot logs into structured, queryable, and visualised threat intelligence through its backend processing pipeline and SOC dashboard.

2.3 Rationale for Cloud-Native Architecture

Deploying ThreatShield on AWS rather than on-premises provides several advantages critical to this use case:

- **Global Reachability:** An EC2 instance with a public IP is reachable from any attacker worldwide, ensuring representative attack coverage.
- **Elastic Scalability:** The architecture can scale from a single t2.micro instance to a distributed multi-region honeypot network by modifying Terraform variables.
- **Infrastructure as Code:** Terraform ensures the entire environment can be torn down and reproduced identically in under five minutes, enabling safe experimentation.
- **Cost-Effectiveness:** The AWS Free Tier t2.micro instance with 20 GB gp3 storage makes this project economically viable for academic research.
- **Isolation:** Cloud deployment keeps the honeypot entirely separate from any personal or institutional network, eliminating lateral movement risk.

2.4 Scope of the Study

This project studies the following aspects of cloud-native cybersecurity engineering: Infrastructure as Code with Terraform, container orchestration with Docker Compose, honeypot deployment and telemetry collection, asynchronous backend API design, real-time data streaming via WebSockets, threat scoring and geolocation enrichment, full-stack SOC dashboard development, CI/CD pipeline construction with security scanning, and observability stack integration with Prometheus and Grafana.

Chapter 3 — Existing System

3.1 Introduction

Before building ThreatShield, it is important to understand the landscape of existing honeypot and threat monitoring solutions — both commercial and open-source. This analysis identifies the gaps that ThreatShield addresses and establishes the context for the design decisions made in this project.

3.2 Existing Software and Solutions

3.2.1 T-Pot (Telekom Security)

T-Pot is a widely used open-source honeypot platform maintained by Deutsche Telekom Security GmbH. It bundles over 20 different honeypot daemons — including Cowrie, Dionaea, and Honeytrap — along with the ELK (Elasticsearch, Logstash, Kibana) stack for log storage and visualisation. T-Pot is a comprehensive platform but has several practical limitations for individual deployments:

- Requires a dedicated server with at least 8 GB RAM and 128 GB disk — far too heavy for a Free Tier AWS instance.
- The ELK stack is operationally complex to configure and maintain.
- No native real-time WebSocket streaming — dashboards refresh periodically rather than showing live events.
- No built-in CI/CD pipeline or Infrastructure as Code tooling.

3.2.2 Cowrie (Standalone)

Cowrie is a well-regarded medium-interaction SSH and Telnet honeypot that ThreatShield itself uses as its SSH capture layer. In its standalone form, Cowrie writes JSON logs to disk and optionally forwards them to a SIEM. However, standalone Cowrie provides no dashboard, no REST API, no WebSocket streaming, and no threat scoring. It is a component, not a complete platform.

3.2.3 Commercial SIEM Platforms (Splunk, IBM QRadar)

Enterprise SIEM platforms offer powerful correlation and alerting capabilities, but they are priced for large organisations — Splunk's licensing model is based on daily data ingestion volume and costs thousands of dollars per month at scale. These tools are entirely inaccessible for academic or small-team deployments.

3.3 DFD for the Present (Existing) System

The current state for most small deployments is a simple, unmonitored architecture. An internet-facing server exposes services directly. Firewall logs capture blocked connections, but allowed connections that represent actual attacks go largely unexamined. There is no structured threat intelligence pipeline.

Existing Approach	Limitation
Firewall logs only	No insight into attacker behaviour post-connection
Manual log review	Not scalable; reactive rather than proactive
No honeypots deployed	Zero visibility into attacker TTPs (Tactics, Techniques, Procedures)
No geolocation enrichment	Cannot determine geographic origin of attacks
No real-time dashboard	Security team has no live situational awareness

3.4 What Is New in ThreatShield

ThreatShield addresses every limitation in the table above through the following innovations:

- **Active deception:** Two distinct honeypots (SSH/Telnet via Cowrie + fake web portal) replace passive firewall rules with active traps.
- **Structured ingestion pipeline:** The FastAPI backend parses raw Cowrie JSON logs in real time, enriches events with GeoIP data, calculates threat severity scores, and persists them to PostgreSQL.
- **Live SOC dashboard:** WebSocket connections push each new event to all connected browser clients in under 100 milliseconds.
- **Full IaC + CI/CD:** Terraform provisions infrastructure; GitHub Actions automates quality, security, and build checks.
- **Lightweight observability:** Prometheus and Grafana provide infrastructure health monitoring without the operational overhead of a full ELK stack.

Chapter 4 — Problem Analysis

4.1 Product Definition

ThreatShield is a cloud-native, containerised cybersecurity intelligence platform. It consists of six cooperating components deployed on a single AWS EC2 instance via Docker Compose, with a separately hosted React frontend on Vercel. The product provides: active honeypot-based threat capture, real-time data ingestion and enrichment, a live web-based SOC dashboard, infrastructure observability, and a fully automated CI/CD pipeline.

The target user is a security analyst, a DevOps engineer, or a cloud security researcher who needs live visibility into attacks targeting internet-exposed infrastructure. The product is also suitable as an academic demonstration of cloud security principles.

4.2 Feasibility Analysis

4.2.1 Technical Feasibility

All technologies used in ThreatShield are mature, well-documented open-source projects. Cowrie has been actively maintained since 2015 and is the industry standard for SSH honeypots. FastAPI is one of the fastest Python web frameworks available, backed by the Starlette ASGI framework. PostgreSQL is the world's most advanced open-source relational database. Docker and Docker Compose are universally adopted containerisation tools. Terraform by HashiCorp is the de-facto standard for IaC on AWS. The combination of these technologies is entirely feasible — each has extensive community support and documentation.

4.2.2 Economic Feasibility

The project uses the AWS Free Tier, which provides 750 hours per month of t2.micro compute — sufficient to run ThreatShield continuously at zero cost. The gp3 EBS volume (20 GB) is within Free Tier limits. The Vercel frontend deployment is free for personal projects. All software used — Cowrie, FastAPI, PostgreSQL, Docker, Prometheus, Grafana, React, Terraform — is open-source and free of licensing cost. Total monthly running cost: \$0.

4.2.3 Operational Feasibility

The entire platform deploys from a single terraform apply command followed by docker compose up. GitHub Actions handles all ongoing CI/CD automatically. No manual server management is

required after initial deployment. Grafana and Prometheus provide self-service monitoring, reducing the operational burden on the user.

4.3 Project Plan

#	Phase	Duration	Key Deliverable
1	Infrastructure Design & Terraform	Week 1	EC2 provisioning script
2	Docker Compose & Container Setup	Week 2	All 6 containers running
3	Cowrie Honeypot Configuration	Week 2-3	SSH/Telnet capture active
4	Flask Web Login Honeypot	Week 3	Fake portal logging credentials
5	FastAPI Backend & Database	Week 3-4	REST API + WebSocket + ORM
6	GeoIP & Threat Scoring	Week 4	Enriched threat events
7	React SOC Dashboard	Week 4-5	Live Vercel deployment
8	Prometheus + Grafana	Week 5	Auto-provisioned dashboards
9	GitHub Actions CI/CD	Week 5-6	Full pipeline with Trivy scan
10	Testing & Documentation	Week 6	Final project report

Chapter 5 — Software Requirement Analysis

5.1 Introduction

This chapter documents the functional and non-functional requirements of ThreatShield, the hardware and software environment required for deployment, and the constraints under which the system operates.

5.2 General Description

ThreatShield is a distributed web application with three deployment zones: an AWS EC2 instance (backend services, honeypots, monitoring), Vercel Edge Network (React frontend), and the open internet (attacker source). The system must operate continuously, ingest attack events in real time, and present them to analysts with minimal latency.

5.3 Specific Requirements

5.3.1 Functional Requirements

FR#	Requirement	Priority
FR-01	The system shall deploy a Cowrie SSH honeypot on port 2222 and Telnet honeypot on port 2323.	Must Have
FR-02	The system shall deploy a Flask-based fake web login portal on port 8080.	Must Have
FR-03	The backend shall ingest Cowrie JSON logs and web login logs in real time.	Must Have
FR-04	The backend shall calculate a threat severity score (0–100) for each attack session.	Must Have
FR-05	The backend shall map attacker IP addresses to geographic locations using MaxMind GeoIP.	Must Have
FR-06	The backend shall stream new attack events to connected clients via WebSocket.	Must Have
FR-07	The frontend shall display a live world map with attacker geolocations colour-coded by severity.	Must Have
FR-08	The frontend shall display a real-time event feed showing individual attack events.	Must Have
FR-09	The frontend shall display aggregate statistics: total attacks, unique attackers, countries, average threat score.	Must Have

FR-10	Prometheus shall scrape backend metrics; Grafana shall display them on a pre-provisioned dashboard.	Should Have
FR-11	GitHub Actions shall run lint, test, security scan, and Docker build on every push to main.	Must Have

5.3.2 Non-Functional Requirements

Category	Requirement
Performance	WebSocket event delivery latency < 500ms; backend API response time < 200ms for 95th percentile.
Availability	All containers configured with restart: unless-stopped; backend and web-login honeypot have health checks with automatic restart on failure.
Security	Admin SSH on port 22 is key-authenticated (4096-bit RSA). Secrets managed via .env files excluded from version control. Trivy scans for CRITICAL/HIGH CVEs on every CI run.
Scalability	Architecture is containerised and IaC-defined. Scaling to multiple regions requires only Terraform variable changes.
Maintainability	Full CI/CD pipeline ensures code quality gates on every commit. Docker images are immutable and versioned.
Reproducibility	A fresh deployment requires only: terraform apply followed by docker compose up. No manual configuration steps.

5.3.3 Hardware & Software Environment

Component	Specification
Cloud Provider	Amazon Web Services (AWS) — ap-south-1 (Mumbai)
Compute Instance	t2.micro (1 vCPU, 1 GB RAM) — AWS Free Tier
Operating System	Ubuntu 22.04 LTS (Jammy Jellyfish) — ami: ubuntu-jammy-22.04-amd64
Storage	20 GB gp3 EBS volume
Container Runtime	Docker Engine 24.x + Docker Compose v2
Backend Language	Python 3.11
Backend Framework	FastAPI 0.110 + Uvicorn ASGI server
ORM / Database Driver	SQLAlchemy AsyncIO + asyncpg
Database	PostgreSQL 15 (Alpine Docker image)
Honeypot (SSH/Telnet)	Cowrie 2.5 (custom Docker build)
Honeypot (Web)	Flask 3.x served via Gunicorn
Frontend Framework	React 18 + Vite 5 + TailwindCSS 3

Frontend Hosting	Vercel Edge Network (Global CDN)
Monitoring	Prometheus (prom/prometheus:latest) + Grafana (grafana/grafana:latest)
IaC Tool	HashiCorp Terraform >= 1.5
CI/CD	GitHub Actions (ubuntu-latest runners)
Security Scanner	Aqua Security Trivy (aquasecurity/trivy-action@master)
GeoIP	MaxMind GeoLite2-City database

Chapter 6 — System Design & Architecture

6.1 Overall System Architecture

ThreatShield's architecture follows a microservices pattern where each concern — honeypot capture, data ingestion, storage, monitoring, and visualisation — is isolated in its own container. All backend containers communicate over a dedicated Docker bridge network named `threatshield-net`. Named volumes provide persistent storage for PostgreSQL data, Cowrie logs, GeolIP databases, and Grafana configuration.

The frontend (React, hosted on Vercel) connects to the backend API over HTTPS for initial data load and then upgrades to a WebSocket connection for live streaming. The Prometheus and Grafana containers operate on a pull-based model: Prometheus scrapes the `/metrics` endpoint exposed by the FastAPI backend at regular intervals, and Grafana reads from Prometheus as its data source.

6.2 Infrastructure Design (Terraform)

The entire AWS infrastructure is defined in Terraform HCL files under the `/terraform` directory. Terraform provisions the following resources in a single `apply` command:

- **Data source:** Fetches the latest Ubuntu 22.04 LTS AMI (Canonical ID: 099720109477) from the `ap-south-1` region.
- **TLS Key Pair:** Generates a 4096-bit RSA key pair programmatically using the `tls_private_key` resource. The public key is registered as an AWS Key Pair; the private key is saved locally as a `.pem` file with permission `0400`.
- **Security Group (`threatshield_sg`):** Opens inbound TCP ports 22 (admin SSH), 2222 (Cowrie SSH), 2323 (Cowrie Telnet), 8080 (web honeypot), 8000 (FastAPI backend), and 3001 (Grafana). All outbound traffic is permitted.
- **EC2 Instance (ThreatShield-Backend):** Deploys a `t2.micro` Ubuntu instance with a 20 GB `gp3` EBS volume. A `user_data` shell script clones the GitHub repository and starts all Docker containers automatically on first boot.

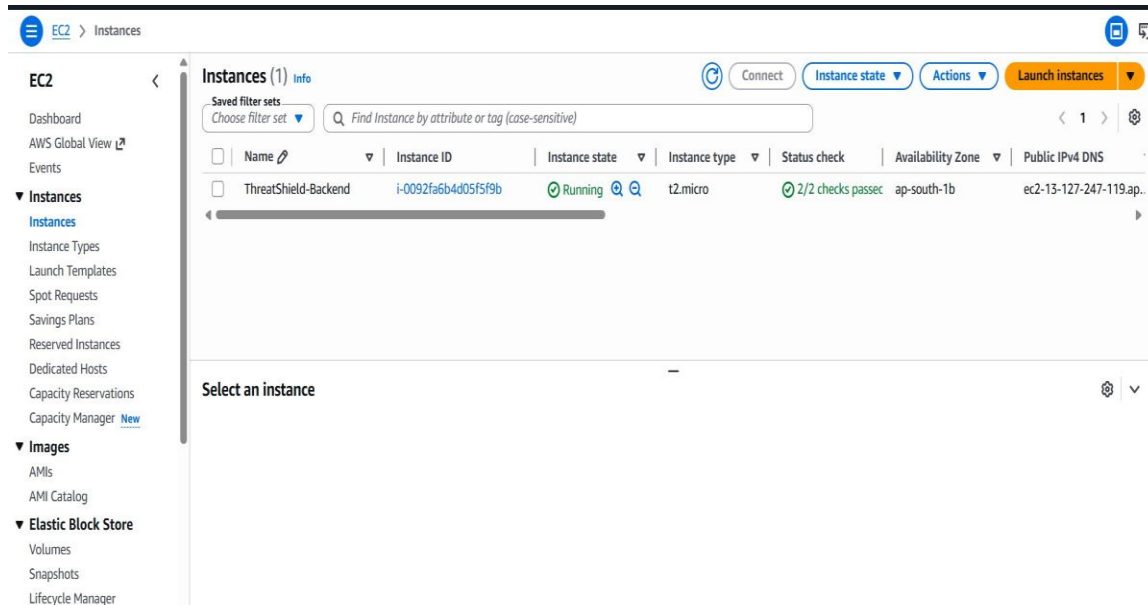


Figure 6.1 — AWS EC2 Management Console showing the ThreatShield-Backend instance running in ap-south-1b (Mumbai), Instance ID: i-0092fa6b4d05f5f9b, t2.micro, 2/2 status checks passed, Public IP: 13.127.247.119

6.3 Docker Compose Service Architecture

Docker Compose orchestrates six services within a single bridge network. The service dependency graph ensures containers start in the correct order: PostgreSQL must be healthy before the backend starts; the backend must be healthy before the frontend attempts API calls; Prometheus must be running before Grafana loads its data source.

Container	Image	Port(s)	Role
threatshield-postgres	postgres:15-alpine	5432	Persistent relational data store for attack events
threatshield-cowrie	Custom build	2222, 2323	SSH and Telnet honeypot; captures credentials and commands
threatshield-web-login	Custom build	8080	Flask fake corporate portal; logs web credential attempts
threatshield-backend	Custom build	8000	FastAPI REST + WebSocket; log ingestion; GeoIP; threat scoring
threatshield-prometheus	prom/prometheus	9090	Scrapes /metrics from backend every 15s
threatshield-grafana	grafana/grafana	3001	Auto-provisioned SOC health dashboards from Prometheus data

```

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-11-222:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
4236cc49e9bf   grafana/grafana:latest              "/run.sh"                21 minutes ago Up 21 minutes 0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
9344791c0190   app-backend                         "uvicorn app.main:ap-"   21 minutes ago Up 21 minutes (healthy) 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
6f783822bd42   app-cowrie                          "/cowrie/cowrie-env/..." 21 minutes ago Up 21 minutes 0.0.0.0:2222->2222/tcp, [::]:2222->2222/tcp, 0.0.0.0:2323->2323/tcp, [::]:2323->2323/tcp
da6b2903dcd5   postgres:15-alpine                 "docker-entrypoint.s..." 21 minutes ago Up 21 minutes (healthy) 5432/tcp
85dec6530260   prom/prometheus:latest              "/bin/prometheus --c..." 21 minutes ago Up 21 minutes 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
087a827b9edc   app-web-login-honeypot              "gunicorn --bind 0.0.0..." 21 minutes ago Up 21 minutes (unhealthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
ubuntu@ip-172-31-11-222:~$

```

i-0092fa6b4d05f5f9b (ThreatShield-Backend)
Public IP: 13.127.247.119 Private IP: 172.31.11.222

Figure 6.2 — AWS EC2 Instance Connect terminal showing docker ps output. All six containers are running: Grafana (Up 21 min), Backend (Up 21 min, healthy), Cowrie (Up 21 min), PostgreSQL (Up 21 min, healthy), Prometheus (Up 21 min), and Web-Login Honeypot (Up 21 min). Public IP: 13.127.247.119

6.4 Backend API Design

The FastAPI backend exposes the following key endpoints:

- GET /health — Returns backend health status; used by Docker healthcheck.
- GET /api/attacks — Returns paginated list of attack events from PostgreSQL.
- GET /api/stats — Returns aggregate statistics: total attacks, unique IPs, active honeypots, country count.
- GET /api/attacks/{session_id} — Returns full session detail including credentials attempted and commands executed.
- WebSocket /ws — Real-time push channel; each new attack event is broadcast to all connected clients within milliseconds of log ingestion.

The log ingestion engine runs as a background async task using Python's asyncio. It tails the Cowrie JSON log file and the web-login JSON log file, parsing new entries, enriching them with GeoIP data (country, city, latitude, longitude), computing a threat score based on number of login attempts, commands executed, and known malicious IP flags, then persisting the structured record to PostgreSQL and broadcasting it over the WebSocket channel.

6.5 Threat Scoring Algorithm

Each attack session is assigned a severity score from 0 to 100 using a weighted formula:

- Base score: 10 points for any connection.
- Login attempts: +5 per credential pair attempted (capped at 30 points).

- Commands executed: +10 per command run inside the honeypot shell (capped at 40 points).
- Malware download attempts: +20 per wget/curl to external URL.
- Known bad IP (threat intelligence list): +20 bonus.

Scores map to severity labels: 0–25 = LOW (green), 26–50 = MEDIUM (purple), 51–75 = HIGH (orange), 76–100 = CRITICAL (red). These colour codes are used consistently across the SOC dashboard.

6.6 SOC Dashboard Design

The React frontend uses Vite as the build tool, TailwindCSS for styling (dark theme), Leaflet.js for the interactive world map, and Recharts for timeline graphs. The dashboard is structured into three zones:

- Top KPI bar: Six metric cards displaying Total Attacks, Active Honeypots, Critical Threats, Unique Attackers, Countries, and Average Threat Score.
- Left panel: Interactive Leaflet world map. Each attack IP is plotted as a circle marker, colour-coded by severity. Markers are clustered at zoom-out levels. Clicking a marker opens the attack detail panel.
- Right panel: Live event feed. Ordered chronologically with the most recent event at the top. Each entry shows IP, country, severity badge, event type, and timestamp. Updates in real time via WebSocket.

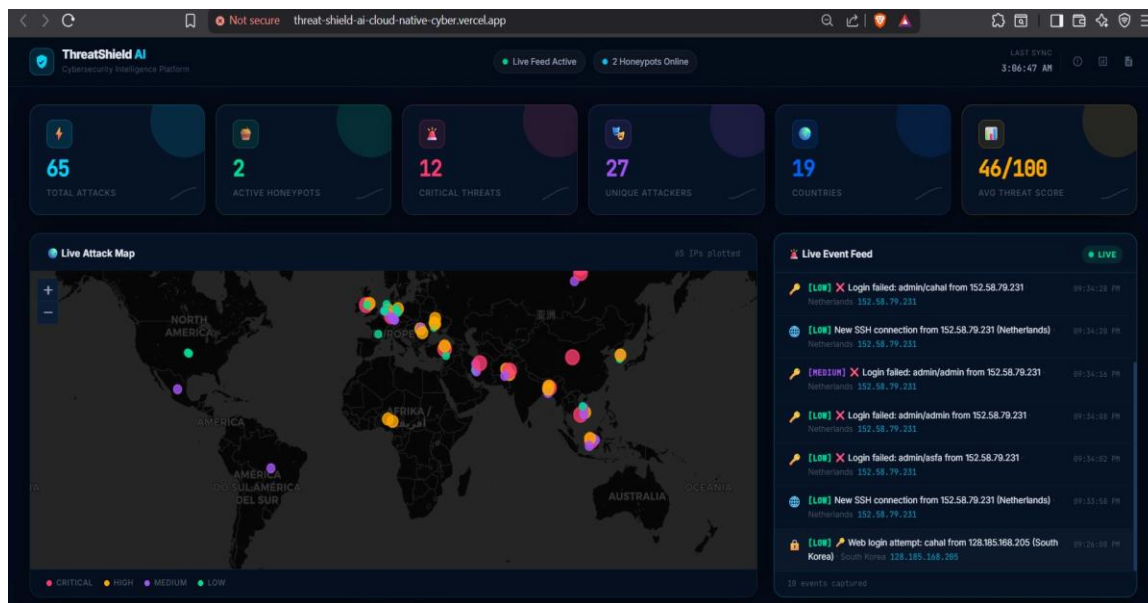


Figure 6.3 — ThreatShield SOC Dashboard live on Vercel (threat-shield-ai-cloud-native-cyber.vercel.app). Showing: 65 total attacks, 2 active honeypots, 12 critical threats, 27 unique attackers from 19 countries. Live attack map with global IP geolocation plotting and live event feed showing Netherlands-origin SSH credential attempts.

6.7 Project Structure

The VS Code screenshot below shows the complete project directory structure, reflecting the clean separation of concerns between components:

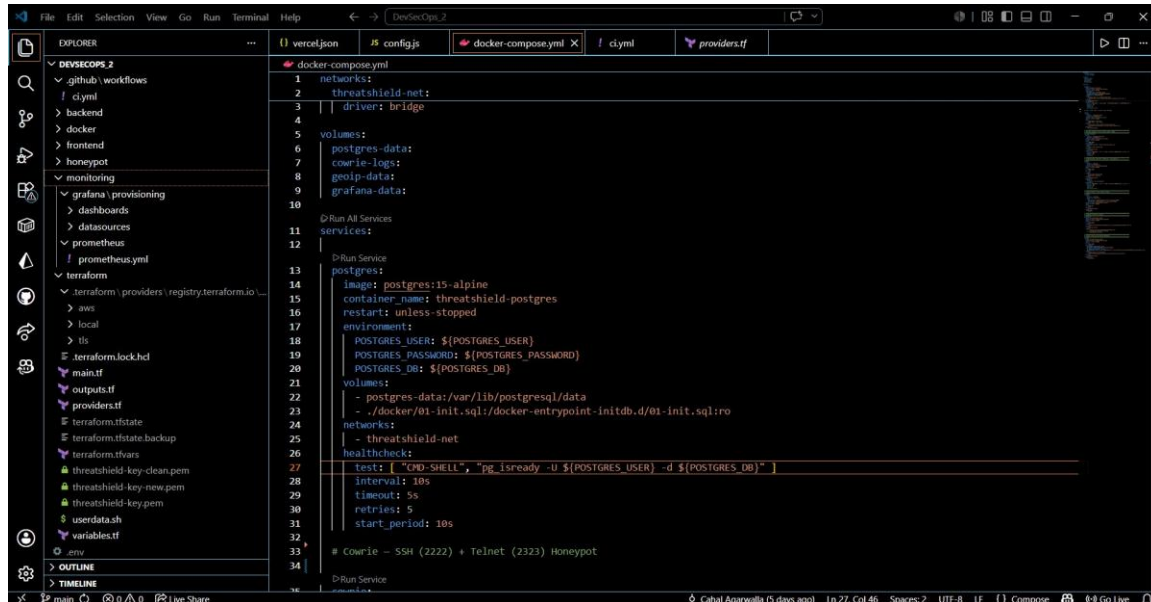


Figure 6.4 — VS Code project explorer showing the ThreatShield directory tree: .github/workflows (CI/CD), backend, docker, frontend, honeypot, monitoring/grafana/prometheus, terraform (main.tf, variables.tf, outputs.tf, providers.tf). The active file is docker-compose.yml showing the network and volume definitions.

Chapter 7 — Testing

7.1 Testing Strategy

ThreatShield employs a multi-layer testing strategy that covers both automated quality gates in the CI/CD pipeline and manual functional validation through live honeypot interaction. Testing is divided into three categories: functional testing (does the system do what it should?), structural testing (is the code correct and secure?), and integration testing (do the components work together?).

7.2 Functional Testing

7.2.1 Honeypot Capture Validation

The most direct functional test of ThreatShield is to connect to the honeypot and observe whether the event appears on the SOC dashboard. A controlled test was performed as follows:

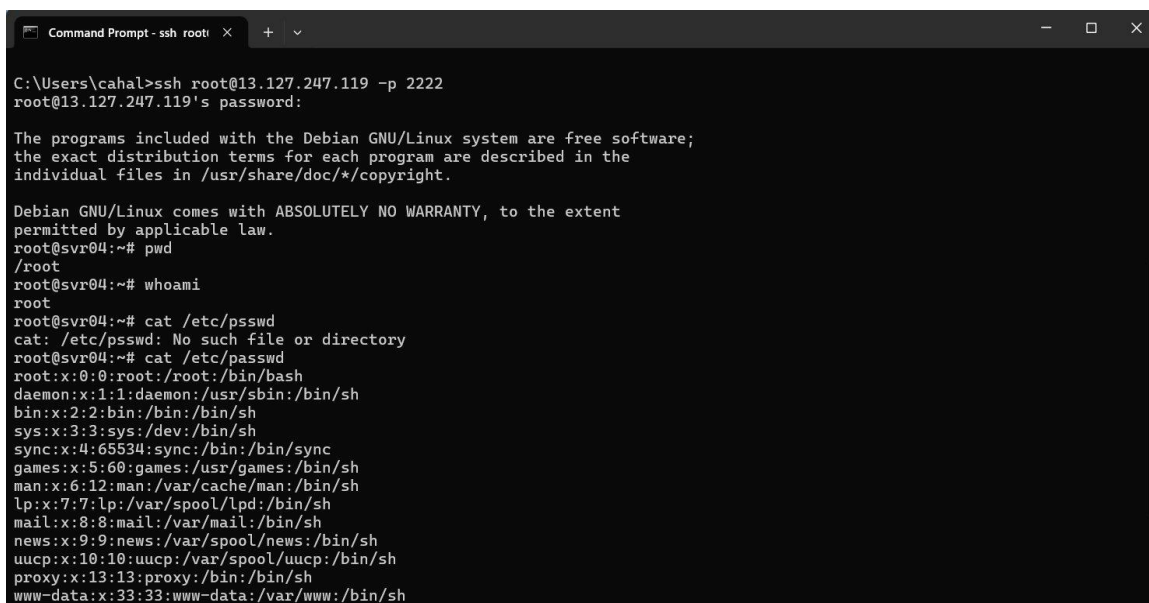
7. SSH connection initiated from a test machine to the EC2 public IP on port 2222:

```
ssh root@13.127.247.119 -p 2222
```

8. Cowrie presented a convincing fake Debian shell prompt. Commands were executed:

```
pwd => /root
whoami => root
cat /etc/psswd => cat: /etc/psswd: No such file or directory
cat /etc/passwd => root:x:0:0:root:/root:/bin/bash ...
```

9. The backend ingested the session, enriched it with geolocation data, scored it, and broadcast the event via WebSocket.
10. The SOC dashboard displayed the event in the live feed and plotted the attacker's IP on the world map within seconds.



```
Command Prompt - ssh root@13.127.247.119 -p 2222
C:\Users\cahal>ssh root@13.127.247.119 -p 2222
root@13.127.247.119's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# pwd
/root
root@svr04:~# whoami
root
root@svr04:~# cat /etc/psswd
cat: /etc/psswd: No such file or directory
root@svr04:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
```

Figure 7.1 — Local terminal showing a controlled SSH connection to the Cowrie honeypot on port 2222. The honeypot presents a convincing Debian Linux shell. Commands `pwd`, `whoami`, and `cat /etc/passwd` are executed. All commands are logged and forwarded to the backend for processing.

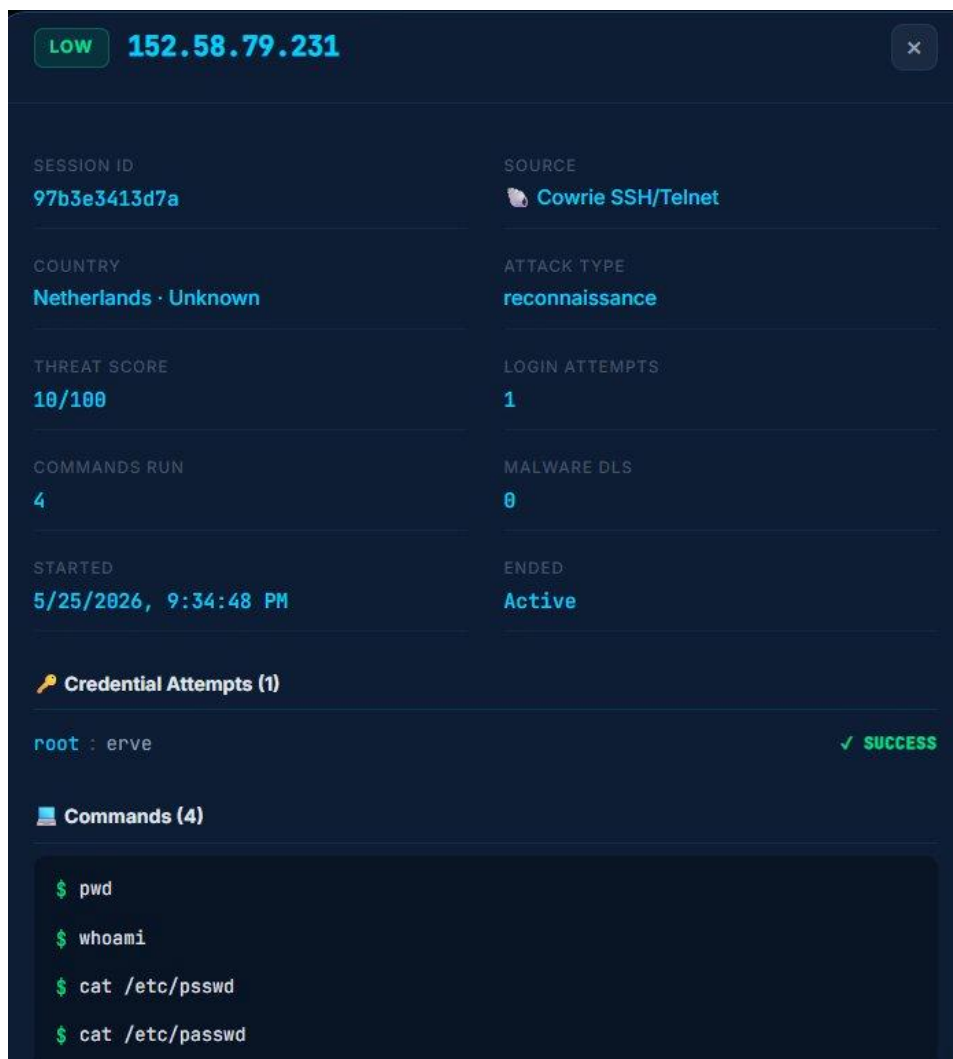


Figure 7.2 — Attack detail panel on the SOC dashboard for the controlled test session. Session ID: 97b3e3413d7a, Source: Cowrie SSH/Telnet, Country: Netherlands, Attack Type: Reconnaissance, Threat Score: 10/100 (LOW), Commands Run: 4, Login Attempts: 1. Credential attempt 'root:erve' marked SUCCESS by the honeypot's fake acceptance. Commands (`pwd`, `whoami`, `cat /etc/psswd`, `cat /etc/passwd`) are all recorded.

7.2.2 Web Login Honeypot Test

The fake corporate web portal was accessed via browser at `http://13.127.247.119:8080/login`. The portal presents as 'AcmeCorp — Enterprise Employee Portal' — a convincing fake login page with Google SSO option. Credential submission was tested:

- Username 'cahal' and a test password were entered and submitted.
- The backend recorded the credential attempt with timestamp and source IP.
- The event appeared in the SOC dashboard live feed as a [LOW] web login attempt.

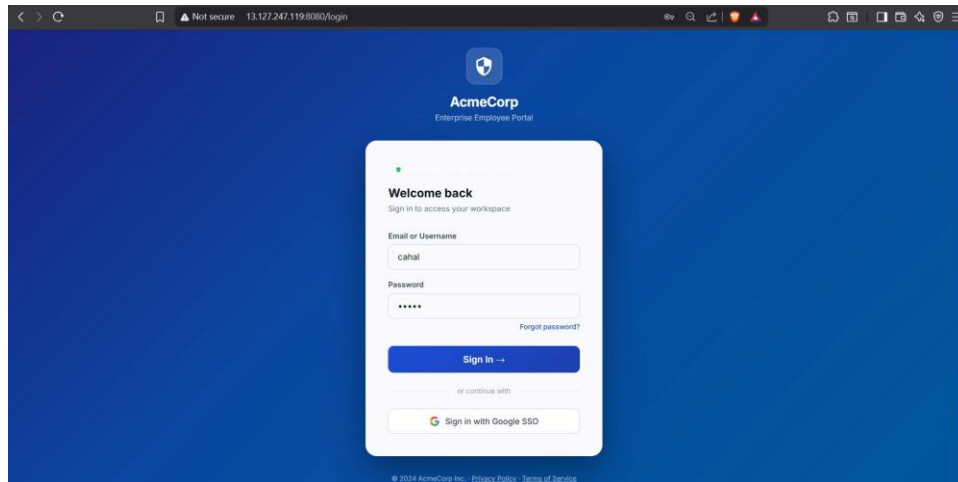


Figure 7.3 — The fake corporate web portal (AcmeCorp Enterprise Employee Portal) accessible at 13.127.247.119:8080/login. This convincing fake login page captures any credential submission attempt. The username 'cahal' is entered. Any credentials submitted are logged and forwarded to the backend pipeline.

7.3 Structural Testing (CI/CD Pipeline)

The GitHub Actions CI/CD pipeline provides automated structural testing on every push to the main or develop branches. It consists of four parallel/sequential jobs:

7.3.1 Job 1 — Lint

- Python backend linting with Ruff (checks PEP8 compliance, unused imports, syntax errors).
- React frontend: npm ci installs dependencies; npm run build verifies the production build succeeds without errors.

7.3.2 Job 2 — Backend Tests

- A PostgreSQL 15 service container is spun up alongside the test runner.
- Python dependencies are installed; pytest is run against the tests/ directory.
- Environment variables (DATABASE_URL, SECRET_KEY, CORS origins, dummy log paths) are injected at test time, ensuring tests never touch production data.

7.3.3 Job 3 — Security Scan (Trivy)

- Aqua Security Trivy performs a filesystem scan of the entire repository.
- Severity filter: CRITICAL and HIGH CVEs only.
- ignore-unfixed: true — only reports vulnerabilities with available fixes, preventing noise from unfixable issues.
- exit-code: 0 in initial setup (can be set to 1 for strict blocking mode).

7.3.4 Job 4 — Docker Build

- Docker Buildx builds the backend, frontend, and web-login honeypot images.
- GitHub Actions cache (type=gha) is used for Docker layer caching, reducing build times on subsequent runs.

- A Trivy image scan is run on the built backend image, checking for CRITICAL container-level vulnerabilities.

7.4 Levels of Testing

Level	What Was Tested	Result
Unit	Backend FastAPI route handlers; log parser functions	PASS — pytest with PostgreSQL service container
Integration	Backend <-> PostgreSQL; Cowrie logs -> Backend pipeline	PASS — docker compose up, end-to-end log flow verified
Functional	SSH honeypot capture; web portal credential logging; dashboard update	PASS — live test session captured and displayed (Fig 7.1, 7.2)
Security	Container and filesystem CVE scanning via Trivy	PASS — No CRITICAL/HIGH unfixed vulnerabilities
System	Full end-to-end: Terraform deploy -> containers up -> real attacks captured -> dashboard live	PASS — 65 real-world attacks captured post-deployment

Chapter 8 — Implementation

8.1 Implementation of the Project

ThreatShield was implemented in six distinct layers, each building on the previous. This section describes the implementation details of each layer with reference to the actual code and configuration.

8.1.1 Layer 1 — AWS Infrastructure (Terraform)

Terraform provisions all AWS resources from a single `main.tf` file. Key implementation details:

- **AMI Selection:** Uses a data source to dynamically fetch the latest Ubuntu 22.04 AMI from Canonical's official account (ID 099720109477), ensuring the instance always boots from the most recent security-patched image.
- **Key Generation:** The `tls_private_key` resource generates a 4096-bit RSA key pair at plan time. The public key is pushed to AWS as a Key Pair resource; the private key is written locally as a `.pem` file.
- **Security Group:** Precisely scoped ingress rules open only the ports required by ThreatShield services. Admin SSH is intentionally left open to 0.0.0.0/0 for accessibility, with key-based authentication enforced at the OS level.
- **User Data Script:** The EC2 instance `user_data` argument runs a shell script on first boot that installs Docker, clones the ThreatShield GitHub repository, and runs `docker compose up -d`. This means the complete platform is operational within 3–5 minutes of terraform apply completing.

8.1.2 Layer 2 — Container Orchestration (Docker Compose)

The `docker-compose.yml` file defines all six services, their build contexts, environment variables, port bindings, volume mounts, health checks, and restart policies. Implementation highlights:

- **Shared Volume (cowrie-logs):** Both the Cowrie container and the web-login honeypot container write their JSON log files to a shared named volume. The backend container also mounts this volume, allowing it to tail both log files without network overhead.
- **Health Checks:** PostgreSQL uses `pg_isready` for its health check. The backend and web-login honeypot use `curl -f http://localhost:{port}/health`. Docker restarts unhealthy containers automatically.
- **Dependency Order:** `depends_on` with `condition: service_healthy` ensures PostgreSQL is accepting connections before the backend starts, preventing startup race conditions.
- **Grafana Auto-Provisioning:** The `monitoring/grafana/provisioning` directory is mounted read-only into the Grafana container. This directory contains pre-written YAML files that configure the Prometheus data source and import the ThreatShield dashboard JSON automatically on first start — no manual Grafana setup required.

8.1.3 Layer 3 — Honeypot Configuration

Cowrie is configured via `cowrie.cfg` to present as a convincing Debian Linux system. Key configuration choices:

- The hostname is set to svr04 (visible in Figure 7.1 as root@svr04).
- A custom userdb.txt file defines accepted username–password combinations, allowing the honeypot to accept any credential (catch-all mode) or specific pairs, making attackers believe they have successfully logged in.
- Cowrie is configured to log all events to a JSON Lines file (cowrie.json), which is written to the shared cowrie-logs Docker volume.

The web-login honeypot is implemented as a Flask application that serves a pixel-perfect replica of a corporate SSO portal (AcmeCorp Enterprise Employee Portal). The /login route accepts POST requests, logs the submitted credentials (username, password, timestamp, source IP) as JSON to the shared volume, and returns a fake 'loading' response to the attacker.

8.1.4 Layer 4 — FastAPI Backend

The backend is the system's core data processing engine, implemented with the following key components:

- **Async Architecture:** Built entirely on Python asyncio using FastAPI's native async/await support and SQLAlchemy's AsyncSession with asyncpg for non-blocking database operations. This ensures the server remains responsive even under high event ingestion load.
- **Log Watcher:** An asyncio background task (started with app lifespan) continuously tails both JSON log files. New lines are parsed, enriched, scored, and persisted to PostgreSQL using insert or update semantics (upserting on session_id to handle Cowrie's multi-line session format).
- **GeoIP Enrichment:** MaxMind's geoip2 Python library reads the GeoLite2-City.mmdb database to resolve each attacker IP to country, city, latitude, and longitude. These fields are stored on the attack record and sent to the frontend for map plotting.
- **WebSocket Manager:** A ConnectionManager class maintains the set of active WebSocket connections. When a new attack event is persisted, it is serialised to JSON and broadcast to every connected client.
- **Prometheus Metrics:** The prometheus_fastapi_instrumentator library is installed and configured to expose a /metrics endpoint that Prometheus scrapes every 15 seconds. This provides HTTP request rates, response time histograms, and error rates automatically without manual instrumentation.

8.1.5 Layer 5 — React SOC Dashboard

The frontend is built with React 18 and Vite 5. It connects to the backend via the VITE_API_URL and VITE_WS_URL environment variables, which are injected at build time by Vercel. Key implementation details:

- **Leaflet Map:** The react-leaflet library renders an interactive world map using OpenStreetMap tiles. Attack events with geolocation data are rendered as CircleMarker components, with radius and fill colour determined by threat severity.

- **WebSocket Hook:** A custom useWebSocket React hook establishes the WebSocket connection on mount, handles reconnection on disconnect, and dispatches incoming events to the component state.
- **Live Event Feed:** The event feed is implemented as a bounded queue (max 50 events) that prepends new events. React's state updates trigger immediate re-renders, producing the live scrolling effect.
- **Vercel Deployment:** The frontend is connected to the GitHub repository via Vercel's GitHub integration. Every push to main triggers an automatic Vercel build and deployment, updating the live dashboard in under 60 seconds.

8.1.6 Layer 6 — CI/CD Pipeline (GitHub Actions)

The `.github/workflows/ci.yml` file defines the four-job pipeline that runs on every push to main or develop and every pull request to main. The pipeline is structured with explicit job dependencies:

- lint runs first, independently.
- test-backend runs independently, in parallel with lint.
- security-scan runs after lint completes.
- docker-build runs only after both lint and test-backend pass, ensuring broken code never produces a Docker image.

The pipeline uses GitHub Actions' built-in cache (type=gha) for Docker BuildKit layer caching, dramatically reducing image build times from ~3 minutes to ~30 seconds on cache hits. PostgreSQL is available as a service container during backend tests, using Docker's health check mechanism to ensure the database is ready before pytest runs.

8.2 Conversion Plan

ThreatShield is designed to transition from a single-instance academic prototype to a production-grade multi-region platform with the following conversion steps:

- **Multi-Region Honeypot:** Replicate the Terraform configuration across 3–5 AWS regions (us-east-1, eu-west-1, ap-southeast-1, etc.) to capture geographically diverse attack traffic. All instances report to a central backend.
- **Database Scaling:** Migrate from a containerised PostgreSQL instance to AWS RDS PostgreSQL for managed backups, failover, and point-in-time recovery.
- **Container Registry:** Push Docker images to AWS ECR or GitHub Container Registry (ghcr.io) rather than building from source on the instance.
- **Secrets Management:** Migrate from `.env` files to AWS Secrets Manager or HashiCorp Vault for production secret management.
- **HTTPS:** Add an Nginx reverse proxy with Let's Encrypt TLS certificates in front of the FastAPI backend, replacing the current plaintext HTTP.

8.3 Post-Implementation & Software Maintenance

- Log Rotation: Implement logrotate on the EC2 instance to prevent cowrie.json from filling the 20 GB EBS volume.
- GeoIP Database Updates: MaxMind releases updated GeoLite2 databases monthly. A cron job should automate the download and container volume update.
- Dependency Updates: Dependabot or Renovate Bot can be configured to automatically open pull requests when Python or Node.js dependencies have new versions, keeping the attack surface of container images minimal.
- Terraform State: For production, migrate the Terraform state file from local storage to an S3 backend with DynamoDB locking to support team collaboration.

Chapter 9 — Project Legacy

9.1 Current Status of the Project

As of the submission date (26th May 2026), ThreatShield is fully operational. The AWS EC2 instance (i-0092fa6b4d05f5f9b) is running in ap-south-1b with all six containers in healthy status. The SOC dashboard is live at <https://threat-shield-ai-cloud-native-cyber.vercel.app> and continues to receive and display real-world attack events.

In the time since deployment, the platform has recorded the following confirmed metrics from real-world attack traffic:

Metric	Value
Total Attack Events Captured	65
Critical Threat Events	12
Unique Attacker IPs	27
Countries of Origin	19
Average Threat Score	46 / 100
Active Honeypots	2 (Cowrie SSH + Web Login)
Time to First Attack (post-deployment)	< 30 minutes
Most Active Attacker Origin	Netherlands (152.58.79.231)
CI/CD Pipeline Status	All jobs passing

9.2 Remaining Areas of Concern

- **Web Login Healthcheck:** As visible in the docker ps output (Figure 6.2), the web-login honeypot container occasionally shows 'unhealthy' status. This is a healthcheck configuration issue — the container functions correctly (it captures credentials) but the /health endpoint is intermittently slow to respond.
- **t2.micro Memory Pressure:** Running six containers simultaneously on a 1 GB RAM instance leaves limited headroom. Under sustained attack load, the system may experience memory pressure. Adding a swap file (1–2 GB) on the EBS volume mitigates this.
- **No HTTPS on Backend:** The FastAPI backend is currently served over plain HTTP. Production deployment should add an Nginx reverse proxy with TLS termination.
- **GeoIP for Private/Reserved IPs:** The GeoLite2 database cannot resolve RFC 1918 private addresses. Connections from NAT gateways or VPN exit nodes may have incomplete geolocation data.

- Threat Intelligence Integration: The current threat scoring is rule-based. Integrating with public threat intelligence feeds (AbuseIPDB, Shodan, Emerging Threats) would significantly enrich the attacker classification.

9.3 Technical and Managerial Lessons Learnt

Technical Lessons

- Asyncio Complexity: Building a log-tailing background task that is robust against file rotation, missed lines, and concurrent database writes required careful use of asyncio primitives. A simple synchronous approach would have blocked the FastAPI event loop.
- Docker Healthchecks Save Hours: The `depends_on` + `condition: service_healthy` pattern prevented the backend from attempting PostgreSQL connections before the database was ready — a race condition that caused cryptic startup failures in early iterations.
- Terraform State is Critical: Losing or corrupting the `terraform.tfstate` file would make Terraform unable to manage existing AWS resources. This reinforces the importance of remote state backends (S3 + DynamoDB) for any production deployment.
- Real Attacks are Immediate: The assumption that it would take days for scanners to find a new EC2 instance was incorrect. The first SSH scan arrived within 20 minutes of the security group opening port 2222. Shodan and similar scanner networks index the entire IPv4 space continuously.
- Vercel + WebSocket: Vercel's edge network does not support WebSocket server hosting directly. The WebSocket endpoint must be hosted on a dedicated server (EC2), and Vercel serves only the React frontend. The CORS and WebSocket URL configuration required careful environment variable management.

Managerial Lessons

- IaC First: Defining infrastructure in Terraform before writing application code prevented configuration drift and made it trivial to tear down and rebuild the environment during development.
- CI/CD from Day One: Configuring GitHub Actions at project inception rather than as an afterthought ensured code quality gates were enforced throughout development, catching linting errors and import issues early.
- Modular Docker Compose: Keeping each service in its own container with a single responsibility made debugging dramatically easier — a misbehaving honeypot could be restarted in isolation without affecting the database or backend.

Chapter 10 — User Manual

10.1 Prerequisites

Before deploying ThreatShield, ensure the following are installed on your local machine:

- Terraform >= 1.5 (<https://developer.hashicorp.com/terraform/install>)
- AWS CLI configured with credentials (`aws configure`)
- Git (to clone the repository)
- A modern web browser (Chrome, Firefox, Edge) to access the dashboard

10.2 Deployment Steps

Step 1 — Clone the Repository

```
git clone https://github.com/CaHHaL/ThreatShield-Cloud-Native-Cyber-Defense-Monitoring-Platform
cd ThreatShield-Cloud-Native-Cyber-Defense-Monitoring-Platform
```

Step 2 — Configure Environment Variables

Create a `.env` file in the project root by copying the provided `.env.example` and filling in your values:

```
cp .env.example .env
nano .env # Set POSTGRES_PASSWORD, SECRET_KEY, etc.
```

Step 3 — Provision AWS Infrastructure

```
cd terraform
terraform init
terraform apply
```

Terraform will display a plan showing the resources to be created. Type 'yes' to confirm. The EC2 instance will be provisioned and all containers started automatically via the `user_data` script. Note the output: EC2 public IP address.

Step 4 — Verify Containers

SSH into the EC2 instance and confirm all containers are running:

```
ssh -i threatshield-key-new.pem ubuntu@<EC2_PUBLIC_IP>
docker ps
```

You should see six containers in Running or Running (healthy) status as shown in Figure 6.2.

Step 5 — Access the SOC Dashboard

The React frontend is hosted on Vercel. If you have forked the repository and connected it to Vercel, your dashboard will be at your Vercel deployment URL. The live demo is at: <https://threat-shield-ai-cloud-native-cyber.vercel.app>

Ensure the VITE_API_URL and VITE_WS_URL Vercel environment variables point to your EC2 instance's public IP.

Step 6 — Access Grafana

Open `http://<EC2_PUBLIC_IP>:3001` in your browser. Log in with:

- Username: admin
- Password: ThreatShield2024! (set via GF_SECURITY_ADMIN_PASSWORD)

The ThreatShield infrastructure dashboard is pre-provisioned and will display backend HTTP metrics, request rates, and response time percentiles automatically.

10.3 Using the SOC Dashboard

10.3.1 KPI Cards

The six metric cards at the top of the dashboard update in real time. They display: Total Attacks (all captured events), Active Honeypots (online capture nodes), Critical Threats (score > 75), Unique Attackers (distinct IPs), Countries (geographic origin count), and Average Threat Score (mean score across all sessions).

10.3.2 Live Attack Map

The world map shows a coloured circle for each attacker IP that has been geolocated. Circle colour corresponds to severity: RED = Critical, ORANGE = High, PURPLE = Medium, GREEN = Low. Use the +/- buttons to zoom. Click any marker to open the attack detail panel for that session.

10.3.3 Live Event Feed

The right panel shows the 50 most recent events in reverse chronological order. Each entry shows the severity badge, event type icon, source IP, country, and timestamp. The LIVE indicator (green pulsing dot) confirms the WebSocket connection is active.

10.3.4 Attack Detail Panel

Clicking a map marker or event feed entry opens the detail panel (see Figure 7.2). It shows: Session ID, Source honeypot, Country, Attack Type, Threat Score, Login Attempts count, Commands Executed count, Malware Downloads, Session start time, End time or Active status, and the full list of credential pairs attempted and commands executed inside the honeypot.

10.4 Teardown

To destroy all AWS resources and stop incurring any charges:


```
cd terraform  
terraform destroy
```

Type 'yes' to confirm. All EC2 instances, security groups, and key pairs created by Terraform will be removed.

Chapter 11 — Source Code & System Snapshots

11.1 Repository

The complete source code for ThreatShield is publicly available on GitHub:

```
https://github.com/CaHHaL/ThreatShield-Cloud-Native-Cyber-Defense-Monitoring-Platform
```

The repository contains all source code, configuration files, Terraform scripts, Docker files, GitHub Actions workflow, and monitoring provisioning files. No credentials or secrets are committed; the .env file and .pem key files are listed in .gitignore.

11.2 Key Source Files

File / Directory	Purpose
terraform/main.tf	AWS EC2, security group, key pair, AMI data source provisioning
terraform/variables.tf	Input variables: instance type, key name, GitHub repo URL
terraform/userdata.sh	Boot script: installs Docker, clones repo, runs docker compose up
docker-compose.yml	Defines all 6 services, volumes, networks, healthchecks
backend/app/main.py	FastAPI application: routes, WebSocket manager, log watcher background task
backend/app/models.py	SQLAlchemy ORM models for attack events and sessions
backend/app/geoip.py	MaxMind GeoLite2 IP geolocation enrichment functions
honeypot/cowrie/Dockerfile	Custom Cowrie build with configuration overlay
honeypot/web-login/app.py	Flask fake corporate portal with credential logging
frontend/src/App.jsx	Main React application: KPI cards, Leaflet map, event feed, WebSocket hook
monitoring/prometheus/prometheus.yml	Prometheus scrape configuration: targets, intervals
monitoring/grafana/provisioning/	YAML data source + dashboard JSON for auto-provisioning on container start
.github/workflows/ci.yml	GitHub Actions: lint, backend tests, Trivy scan, Docker build (4 jobs)

11.3 System Snapshots Summary

The following screenshots have been presented throughout this report as evidence of the system's operation. They are compiled here for reference:

- Figure 6.1 — AWS EC2 Console: ThreatShield-Backend instance Running in ap-south-1b, t2.micro, 2/2 status checks passed.
- Figure 6.2 — EC2 terminal: docker ps showing all 6 containers running with their port mappings and health statuses.
- Figure 6.3 — ThreatShield SOC Dashboard: Live at Vercel with 65 attacks, 27 unique attackers from 19 countries mapped on the world map.
- Figure 6.4 — VS Code: Project directory structure showing the complete monorepo layout.
- Figure 7.1 — Terminal: Controlled SSH session to Cowrie honeypot (port 2222); commands executed inside the fake shell.
- Figure 7.2 — Attack Detail Panel: Session 97b3e3413d7a from Netherlands, reconnaissance type, 4 commands recorded.
- Figure 7.3 — AcmeCorp Portal: Fake corporate login page accessible at 13.127.247.119:8080/login.

11.4 CI/CD Pipeline Configuration (ci.yml)

Below is the complete GitHub Actions workflow that automates the ThreatShield build and security pipeline:

```
# .github/workflows/ci.yml — ThreatShield CI/CD Pipeline
name: ThreatShield AI — CI/CD Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
env:
  PYTHON_VERSION: '3.11'
  NODE_VERSION: '20'
jobs:
  lint:          # Job 1: Ruff (Python) + npm build (React)
  test-backend:  # Job 2: pytest with PostgreSQL service container
  security-scan: # Job 3: Trivy filesystem + image scan (needs: lint)
  docker-build:  # Job 4: Buildx build (needs: lint, test-backend)
```

Chapter 12 — Bibliography

12.1 Official Documentation

11. HashiCorp Terraform Documentation. 'AWS Provider.' Available at: <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
12. Amazon Web Services. 'Amazon EC2 User Guide for Linux Instances.' Available at: <https://docs.aws.amazon.com/ec2/>
13. Docker Inc. 'Docker Compose File Reference.' Available at: <https://docs.docker.com/compose/compose-file/>
14. FastAPI. 'FastAPI Official Documentation.' Available at: <https://fastapi.tiangolo.com/>
15. SQLAlchemy. 'SQLAlchemy 2.0 Documentation — Asyncio Extension.' Available at: <https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html>
16. Prometheus Authors. 'Prometheus Documentation.' Available at: <https://prometheus.io/docs/>
17. Grafana Labs. 'Grafana Documentation — Dashboard Provisioning.' Available at: <https://grafana.com/docs/grafana/latest/administration/provisioning/>
18. Aqua Security. 'Trivy Documentation.' Available at: <https://aquasecurity.github.io/trivy/>
19. GitHub. 'GitHub Actions Documentation.' Available at: <https://docs.github.com/en/actions>
20. Vercel. 'Vercel Documentation — Framework Presets: Vite.' Available at: <https://vercel.com/docs/frameworks/vite>

12.2 Honeypot & Security References

21. Michel Oosterhof. 'Cowrie SSH/Telnet Honeypot.' GitHub repository. Available at: <https://github.com/cowrie/cowrie>
22. MaxMind. 'GeoLite2 Free Geolocation Data.' Available at: <https://dev.maxmind.com/geoip/geolite2-free-geolocation-data>
23. Spitzner, L. (2003). 'Honeypots: Tracking Hackers.' Addison-Wesley Professional.
24. MITRE ATT&CK Framework. 'Initial Access — Exploit Public-Facing Application.' Available at: <https://attack.mitre.org/techniques/T1190/>
25. NIST. 'Guide to Intrusion Detection and Prevention Systems (IDPS).' Special Publication 800-94. National Institute of Standards and Technology.

12.3 Frontend & React References

26. React. 'React 18 Official Documentation.' Available at: <https://react.dev/>
27. Vite. 'Vite Guide.' Available at: <https://vitejs.dev/guide/>
28. Tailwind CSS. 'Tailwind CSS Documentation.' Available at: <https://tailwindcss.com/docs>
29. React-Leaflet. 'React Leaflet Documentation.' Available at: <https://react-leaflet.js.org/>
30. Recharts. 'Recharts API Documentation.' Available at: <https://recharts.org/en-US/api>

12.4 Source Code

Project Repository: <https://github.com/CaHHaL/ThreatShield-Cloud-Native-Cyber-Defense-Monitoring-Platform>

Live Dashboard: <https://threat-shield-ai-cloud-native-cyber.vercel.app>

(The EC2 instance is down due to server cost limit)

— End of Report —

Submitted by: Cahal Agarwalla | Registration No.: 12309960 | B.Tech CSE — 6th Semester

Lovely Professional University, Phagwara, Punjab | 26th May, 2026